

Inigo Quilez :: articles :: gpu conditionals - 2025



Intro

In this article I want to correct a popular misconception that's been making the rounds in computer graphics aficionado circles for a long time now. It has to do with conditionals when selecting between two results in the GPUs. Unfortunately there are a couple of educational websites out there that are spreading some misinformation, and it would be nice correcting that. I tried contacting the authors without success, so without further ado, here goes my attempt to fix things up a little:

The issue

So, say I have this code, which I actually published the other day:

```
vec2 snap45( in vec2 v )
{
    vec2 s = sign(v);
    float x = abs(v.x);
    return x>0.923880?vec2(s.x,0.0):
           x>0.382683?s*sqrt(0.5):
           vec2(0.0,s.y);
}
```

The exact details of what it does don't matter for this discussion. All we care about is the two ternary operations deciding what's the final value this function should return. Indeed, depending on the value of the variable **x**, the function will return one of three results, which are simple to compute. I could also have implemented this function with regular **if** statements, and all that I'm going to say in this article stays true.

Now, here's the problem - when seeing code like this, somebody somewhere will step up and invariably propose the following "optimization", which replaces what they believe (erroneously) are "conditional branches" in the code, by arithmetic operations. They will suggest something like this:

```

vec2 snap45( in vec2 v )
{
    vec2 s = sign(v);
    float x = abs(v.x);

    float w0 = step(0.92387953,x);
    float w1 = step(0.38268343,x)*(1.0-w0);
    float w2 = 1.0-w0-w1;

    vec2 res0 = vec2(s.x,0.0);
    vec2 res1 = vec2(s.x,s.y)*sqrt(0.5);
    vec2 res2 = vec2(0.0,s.y);

    return w0*res0 + w1*res1 + w2*res2;
}

```

There are two things wrong with this practice. The first one shows an incorrect understanding of how the GPU works. In particular, the original shader code had no conditional branching in it. Selecting between a few registers with a ternary operator or with a plain `if` statement does not lead to conditional branching; all it involves is a conditional move (a.k.a. "select"), which is a simple instruction to route the correct bits to the destination register. You can think of it as a bitwise AND+NAND+OR on the source registers, which is a simple combinational circuit. I'll repeat it again - there is no branching, the instruction pointer isn't manipulated, there's no prediction involved, no pipe to flush, no instruction cache to invalidation, no nothing.

For the record, of course GPUs can do real branching, and those are fine and fast and totally worth it when big chunks of code and computation are to be skipped given a condition. As with all things computing, always check the generated machine code to know what is happening exactly and when. But one thing you can safely assume without having to check any generated code - when moving simple values or computations like in my original example, you are guaranteed to not branch. This has been true for decades at this point, with GPUs. And while I'm not an expert in CPUs, I am pretty sure this is true for them as well.

The second wrong thing with the supposedly optimized version is that it actually runs much slower than the original version. You can measure it in a variety of hardware. I can only assume that's because the `step()` function is probably implemented with some sort of conditional move or subtract + bit propagation + AND.

```

float step( float x, float y )
{
    return x < y ? 1.0 : 0.0;
}

```

Either way, using the `step()` "optimization" are either using the ternary operation anyways, which produces the 0.0 or 1.0 which they will use to mask in and out the different potential outputs with a series of arithmetic

multiplications and additions. Which is wasteful, the values could have been conditionally moved directly, which is what the original shader code did.

But don't take my word for it, let's look at the generated machine code for the original code I published:

GLSL

```
return x>0.923880?vec2(s.x,0.0):
      x>0.382683?s*sqrt(0.5):
      vec2(0.0,s.y);
```

AMD Compiler

```
s_mov_b32      s0, 0x3ec3ef15
v_mul_f32      v3, 0x3f3504f3, v1
v_mul_f32      v4, 0x3f3504f3, v0
s_mov_b32      s1, 0x3f6c835e
v_cmp_gt_f32    vcc, abs(v2), s0
v_cndmask_b32   v3, 0, v3, vcc
v_cndmask_b32   v0, v0, v4, vcc
v_cmp_ngt_f32   vcc, abs(v2), s1
v_cndmask_b32   v1, v1, v3, vcc
v_cndmask_b32   v0, 0, v0, vcc
```

Microsoft Compiler

```
lt    r0.xy, l(0, 0), v0.xy
lt    r0.zw, v0.xy, l(0, 0)
iadd  r0.xy, -r0.xyxx, r0.zwzz
itof  r0.xy, r0.xyxx
mul   r1.xyzw, r0.xyxy, l4(0.707107)
lt    r2.xy, l(0.923880,0.382683), |v0.xx|
mov   r0.z, l(0)
movc  r1.xyzw, r2.yyyy, r1.xyzw, r0.zyzy
movc  o0.xyzw, r2.xxxx, r0.xzzz, r1.xyzw
```

Here we can confirm that the GPU is not branching, as I explained. Instead, according to the AMD compiler, it's performing the required comparisons (`v_cmp_gt_f32` and `v_cmp_ngt_f32` - cmp=compare, gt=greater than, ngt=not greater than), and then using the result to mask the results with the bitwise operations mentioned earlier (`v_cndmask_b32` - cnd=conditional).

The Microsoft compiler has expressed the same idea/implementation in a different format, but you can still see the comparison (`lt` - "lt"=less than) and the masking or conditional move (`movc` - mov=move, c=conditionally).

There are no jump/branch instructions in these listings.

Something not related to the discussion but interesting, is that some of the `abs()` GLSL calls I had in my shader before the ternary operator we are discussing, didn't become GPU instructions but rather instruction modifiers, which is the reason you see them in the listing. This means you can think of `abs()` calls as being

free.

Conclusion

So, if you ever see somebody proposing this

```
float a = mix( b, c, step( y, x ) );
```

as an optimization to

```
float a = x < y ? b : c;
```

then please correct them for me.

Thanks!

inigo quilez - learning computer graphics since 1994